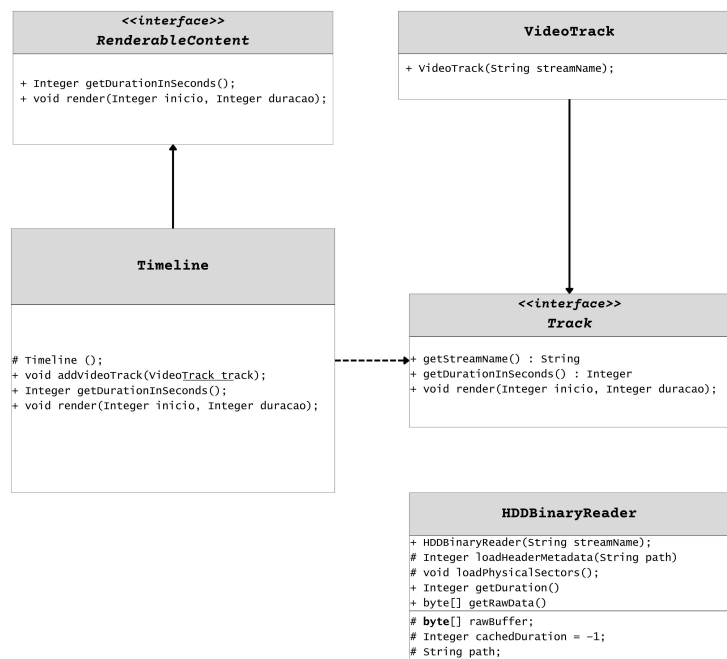


Seguindo o desenvolvimento do framework de vídeo, uma grande distribuidora de streaming está disposta a utilizar o framework, tendo solicitado, contudo, uma nova funcionalidade para a distribuição dos vídeos construídos. O protocolo de distribuição de streaming faz uso de dois métodos a buferização e construção dos datagramas para a distribuição dos filmes:

Integer getDurationInSeconds() - Método que informa a duração total do filme.

render(Integer inicio, Integer duracao) - Método que renderiza um trecho específico do filme iniciando no tempo dado por início e com duração dada por duracao.

Seguindo o princípio da segregação de interface, estes métodos foram especificados na interface **RenderableContent**, que agora é implementada por **Timeline**. O diagrama parcial da solução é apresentado à seguir, com destaque para a interface **RenderableContent**.



Questão I

Para a operadora de streaming é importante a capacidade de customizar dinamicamente o fluxo final de exibição de uma **Timeline** finalizada. Dependendo do perfil de assinatura ou da região do usuário conectado, o sistema precisa injetar automaticamente um ou mais **Trailers** (renderizado antes do início do vídeo principal) e/ou uma ou mais tela de **Créditos** customizada (renderizado imediatamente após o término do conteúdo principal). A operadora chamou a atenção que a combinação desses elementos periféricos de mídia é altamente volátil e imprevisível: uma mesma Timeline (filme) pode ser exibida pura, apenas com créditos no fim, ou com trailer na abertura mais créditos embutidos. Para a classe Stream, que executa o protocolo de distribuição, a montagem dessa linha do tempo estendida deve se comportar de forma transparente, não importando se existe ou não trailers ou créditos no filme original. Adicionalmente, a solução encontrada deve permitir a incorporação futura de novos elementos, como avisos de classificação indicativa ou propagandas, sem que isso resulte em explosão da complexidade de gestão de classificadores do framework. Proponha e implemente uma solução baseada em padrões de projeto que enderece os requisitos apontados pela operadora de streaming.

Questão II

O sistema de arquivos da operadora utiliza uma classe nativa de acesso aos arquivos físicos, a classe `HDDBinaryReader`. A classe tem seu esqueleto de código fornecida em anexo. Um detalhe importante de funcionamento da classe é que o seu método público `getRawData()` realiza uma cópia profunda e redundante do buffer nativo a cada vez que é invocada para evitar que eventuais modificações realizadas em alguns processos sejam vistos por outro, antes de ser tornarem consistentes. Para o nosso framework, que opera com muitos arquivos de grande tamanho, isto tem sido um gargalo operacional. A equipe observou também que a estrutura interna `protected byte[] rawBuffer` e o método de ativação `protected void loadPhysicalSectors()` estão expostos com visibilidade protegida.

A única forma de acessar os arquivos que formarão as diversas tracks usadas pelo framework é através da classe **`HDDBinaryReader`**. É sua tarefa usar padrões de projeto para resolver esta questão, sem contudo duplicar o consumo de memória, resultante da chamada de `getRawData()`, quando a carga dos bytes de conteúdo se faz necessária.

```
package br.edu.ifba.inf011.aval.thirdPart;

import java.util.Random;

public class HDDBinaryReader {

    protected byte[] rawBuffer;
    protected Integer cachedDuration = -1;
    protected String path;

    /*
     * Inicializa um HDDBinaryReader com o nome do arquivo em disco e armazena em path.
     * Chama o método loadHeaderMetadata para carregar os metadados do arquivo (inclusive tamanho)
     * Os bytes do arquivo não são carregados.
     */
    public HDDBinaryReader(String path){...}

    /*
     * Carrega os metadados do arquivo e armazena o tamanho em cachedDuration
     */
    protected Integer loadHeaderMetadata(String path) {...}

    /*
     * Carrega os bytes com os dados do arquivo e armazena em rawBuffer
     */
    protected void loadPhysicalSectors(String path) {...}}

    /*
     * Retorna o metadado de duração do arquivo
     */
    public Integer getDuration() {...}

    /*
     * O método verifica se os dados já foram carregados para a memória. Caso não tenha sido
     * chama loadPhysicalSectors() para fazer a carga. Em seguida, a cada chamada,
     * faz uma cópia dos dados em rawBuffer e devolve para quem solicitou.
     */
    public byte[] getRawData() {...}

}
```